УО «Белорусский государственный университет информатики и радиоэлектроники»
Кафедра ПОИТ

Отчет по лабораторной работе №1
по предмету
Распределенные вычисления
Вариант 3

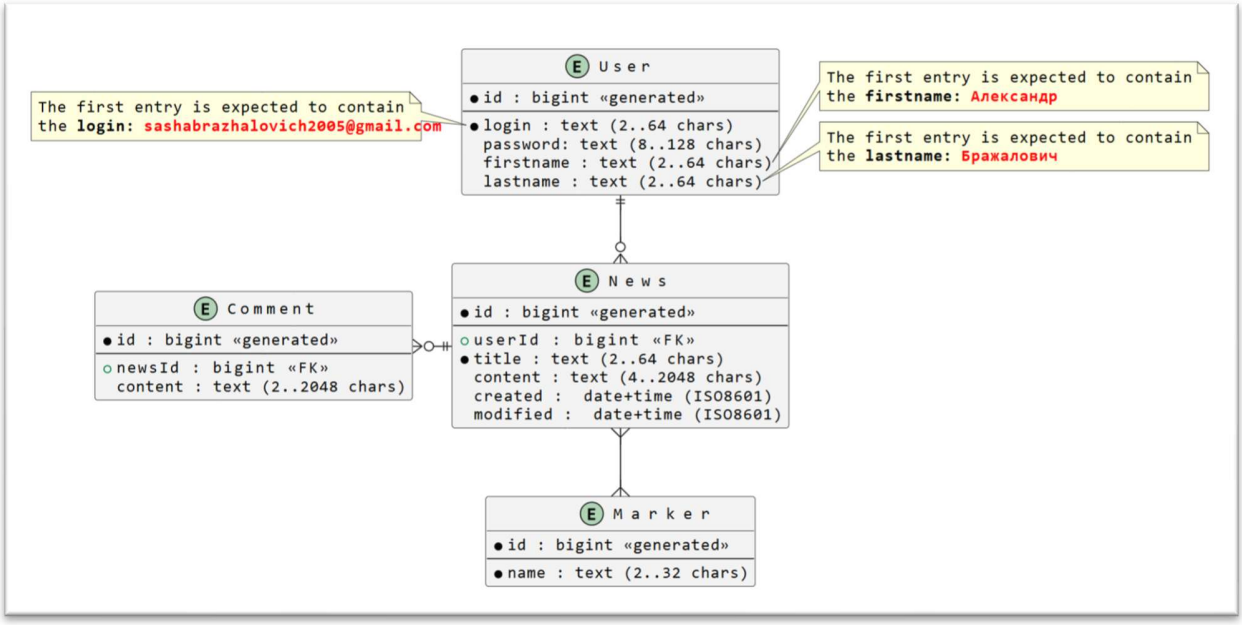
Выполнил:
Бражалович А.И.

Проверил:
Данилова Г.В.

Группа 351004

Минск 2026

Задание



Код

```
package org.example.newsapi.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.CommentRequestTo;
import org.example.newsapi.dto.response.CommentResponseTo;
import org.example.newsapi.service.CommentService;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1.0/comments")
@RequiredArgsConstructor
public class CommentController {

    private final CommentService commentService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public CommentResponseTo create(@RequestBody @Valid CommentRequestTo
request) {
        return commentService.create(request);
    }
```

```
    @GetMapping
    public List<CommentResponseTo> getAll(@PageableDefault(size = 50)
Pageable pageable) {
        return commentService.findAll(pageable).getContent();
    }

    @GetMapping("/{id}")
    public CommentResponseTo getById(@PathVariable Long id) {
        return commentService.findById(id);
    }

    @PutMapping("/{id}")
    public CommentResponseTo update(@PathVariable Long id, @RequestBody
@Valid CommentRequestTo request) {
        return commentService.update(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        commentService.delete(id);
    }
}
```

```
package org.example.newsapi.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.MarkerRequestTo;
import org.example.newsapi.dto.response.MarkerResponseTo;
import org.example.newsapi.service.MarkerService;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1.0/markers")
@RequiredArgsConstructor
public class MarkerController {

    private final MarkerService markerService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public MarkerResponseTo create(@RequestBody @Valid MarkerRequestTo
request) { // ПРОВЕРЬ @Valid
        return markerService.create(request);
    }

    @GetMapping
    public List<MarkerResponseTo> getAll(@PageableDefault(size = 50) Pageable
pageable) {
        return markerService.findAll(pageable).getContent();
    }
```

```
    @GetMapping("/{id}")
    public MarkerResponseTo getById(@PathVariable Long id) {
        return markerService.findById(id);
    }

    @PutMapping("/{id}")
    public MarkerResponseTo update(@PathVariable Long id, @RequestBody @Valid
MarkerRequestTo request) {
        return markerService.update(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        markerService.delete(id);
    }
}
package org.example.newsapi.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.NewsRequestTo;
import org.example.newsapi.dto.response.NewsResponseTo;
import org.example.newsapi.service.NewsService;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1.0/news")
@RequiredArgsConstructor
public class NewsController {

    private final NewsService newsService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public NewsResponseTo create(@RequestBody @Valid NewsRequestTo request) {
        System.out.println(">>> CREATE NEWS REQUEST: " + request);
        System.out.println(">>> markerNames: " + request.getMarkerNames());
        return newsService.create(request);
    }

    @GetMapping
    public List<NewsResponseTo> getAll(@PageableDefault(size = 50) Pageable
pageable) {
        return newsService.findAll(pageable).getContent();
    }

    @GetMapping("/{id}")
    public NewsResponseTo getById(@PathVariable Long id) {
        return newsService.findById(id);
```

```
    }

    @PutMapping("/{id}")
    public NewsResponseTo update(@PathVariable Long id, @RequestBody @Valid
NewsRequestTo request) {
        return newsService.update(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        newsService.delete(id);
    }
}
package org.example.newsapi.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.UserRequestTo;
import org.example.newsapi.dto.response.UserResponseTo;
import org.example.newsapi.service.UserService;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1.0/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public UserResponseTo create(@RequestBody @Valid UserRequestTo request) {
        return userService.create(request);
    }

    @GetMapping
    public List<UserResponseTo> getAll(@PageableDefault(size = 50) Pageable
pageable) {
        // Возвращаем только контент списка, чтобы тестер не путался в мета-
данных Page
        return userService.findAll(pageable).getContent();
    }

    @GetMapping("/{id}")
    public UserResponseTo getById(@PathVariable Long id) {
        return userService.findById(id);
    }

    @PutMapping("/{id}")
    public UserResponseTo update(@PathVariable Long id, @RequestBody @Valid
```

```
UserRequestTo request) {
        return userService.update(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        userService.delete(id);
    }
}
package org.example.newsapi.dto.request;

import com.fasterxml.jackson.annotation.JsonProperty;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.Data;

@Data
public class CommentRequestTo {

    //@JsonProperty("news")
    @NotNull
    private Long newsId;

    @Size(min = 2, max = 2048)
    private String content;

    public void setNews(Long news) {
        this.newsId = news;
    }
}
package org.example.newsapi.dto.request;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor // Обязательно для правильной работы Jackson
@AllArgsConstructor
public class MarkerRequestTo {
    @NotBlank
    @Size(min = 2, max = 32)
    private String name;
}
package org.example.newsapi.dto.request;

import com.fasterxml.jackson.annotation.JsonProperty;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.Data;
import java.util.Set;

@Data
public class NewsRequestTo {
```

```
    @NotNull
    private Long userId;

    @Size(min = 2, max = 64)
    private String title;

    @Size(min = 4, max = 2048)
    private String content;

    // Это поле будет заполняться через сеттеры ниже
    private Set<String> markerNames;

    // Сеттер для JSON-поля "marker"
    public void setMarker(Set<String> markerNames) {
        System.out.println(">>> setMarker called with: " + markerNames);
        this.markerNames = markerNames;
    }

    // Сеттер для JSON-поля "markers" (на случай множественного числа)
    public void setMarkers(Set<String> markerNames) {
        System.out.println(">>> setMarkers called with: " + markerNames);
        this.markerNames = markerNames;
    }
}
package org.example.newsapi.dto.request;

import jakarta.validation.constraints.Size;
import lombok.Data;

@Data
public class UserRequestTo {
    @Size(min = 2, max = 64)
    private String login;

    @Size(min = 8, max = 128)
    private String password;

    @Size(min = 2, max = 64)
    private String firstname;

    @Size(min = 2, max = 64)
    private String lastname;
}
package org.example.newsapi.dto.response;

import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.Data;

@Data
public class CommentResponseTo {

    private Long id;

    //@JsonProperty("news")
    private Long newsId;

    private String content;
```

```
    public Long getNews() {
        return this.newsId;
    }
}
package org.example.newsapi.dto.response;

import lombok.Data;

@Data
public class MarkerResponseTo {
    private Long id;
    private String name;
}
package org.example.newsapi.dto.response;

import com.fasterxml.jackson.annotation.JsonAlias;
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.Data;
import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;

@Data
public class NewsResponseTo {
    private Long id;


    //@JsonProperty("user")
    //@JsonAlias({"userId", "user"})
    private Long userId;

    private String title;
    private String content;
    private LocalDateTime created;
    private LocalDateTime modified;

    //@JsonProperty("marker")
    private Set<Long> markerIds = new HashSet<>();

    public Long getUser() {
        return this.userId;
    }

    // Jackson создаст поле "marker" в JSON ответе
    public Set<Long> getMarker() {
        return this.markerIds;
    }
}
package org.example.newsapi.dto.response;

import com.fasterxml.jackson.annotation.JsonTypeName;
import com.fasterxml.jackson.annotation.JsonTypeInfo;
import com.fasterxml.jackson.annotation.JsonRootName;
import lombok.Data;

@Data
```

```java
@JsonRootName("user") // Если тест требует обертку
public class UserResponseTo {
    private Long id;
    private String login;
    private String firstname;
    private String lastname;
}
package org.example.newsapi.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "tbl_comment")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Comment {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"comment_seq_gen")
    @SequenceGenerator(name = "comment_seq_gen", sequenceName =
"comment_seq", allocationSize = 1)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "news_id", nullable = false)
    @ToString.Exclude
    private News news;

    @Column(nullable = false, length = 2048)
    private String content;
}
package org.example.newsapi.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "tbl_marker")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Marker {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"marker_seq_gen")
    @SequenceGenerator(name = "marker_seq_gen", sequenceName = "marker_seq",
allocationSize = 1)
    private Long id;

    @Column(unique = true, nullable = false, length = 32)
```

```
    private String name;
}
package org.example.newsapi.entity;

import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;
import org.hibernate.annotations.OnDelete;
import org.hibernate.annotations.OnDeleteAction;

import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

@Entity
@Table(name = "tbl_news")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class News {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"news_seq_gen")
    @SequenceGenerator(name = "news_seq_gen", sequenceName = "news_seq",
allocationSize = 1)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    // ВАЖНО: это удалит новость, когда удаляется юзер
    @org.hibernate.annotations.OnDelete(action =
org.hibernate.annotations.OnDeleteAction.CASCADE)
    private User user;

    @Column(nullable = false, unique = true, length = 64)
    private String title;

    @Column(nullable = false, length = 2048)
    private String content;

    @CreationTimestamp
    private LocalDateTime created;

    @UpdateTimestamp
    private LocalDateTime modified;

    @Builder.Default
    @ManyToMany(fetch = FetchType.EAGER, cascade = CascadeType.ALL)
    @JoinTable(
            name = "tbl_news_marker",
            joinColumns = @JoinColumn(name = "news_id"),
            inverseJoinColumns = @JoinColumn(name = "marker_id")
    )
```

```
    private Set<Marker> markers = new HashSet<>();

    @OneToMany(mappedBy = "news", cascade = CascadeType.ALL, orphanRemoval =
true)
    @ToString.Exclude
    private List<Comment> comments;
}
package org.example.newsapi.entity;

import jakarta.persistence.*;
import lombok.*;

import java.util.List;

@Entity
@Table(name = "tbl_user")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"user_seq_gen")
    @SequenceGenerator(name = "user_seq_gen", sequenceName = "user_seq",
allocationSize = 1)
    private Long id;

    @Column(nullable = false, unique = true, length = 64)
    private String login;

    @Column(nullable = false, length = 128)
    private String password;

    @Column(length = 64)
    private String firstname;

    @Column(length = 64)
    private String lastname;

    // Связь один-ко-многим с новостями
    // mappedBy указывает на поле "user" в классе News
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch =
FetchType.LAZY)
    @ToString.Exclude // Важно! Исключаем из toString, чтобы не было рекурсии
и переполнения стека
    private List<News> news;
}
package org.example.newsapi.exception;

public class AlreadyExistsException extends RuntimeException {
    public AlreadyExistsException(String message) {
        super(message);
    }
}
```

```java
package org.example.newsapi.exception;
import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class ErrorResponse {
    private String errorMessage;
    private int errorCode; // 5 digits
}
package org.example.newsapi.exception;

import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // 1. БИЗНЕС-ОШИБКИ (Не найдено или Дубликат логина/заголовка) -> 403
    // Объединяем их в один метод, чтобы не было конфликтов

    @ExceptionHandler({NotFoundException.class,
AlreadyExistsException.class})
    public ResponseEntity<ErrorResponse> handleBusinessError(RuntimeException
e) {
        System.out.println(">>> HANDLED BUSINESS ERROR: " + e.getMessage());
// ЛОГ ДЛЯ НАС
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
                .body(new ErrorResponse(e.getMessage(), 40301));
    }


@ExceptionHandler(org.springframework.dao.DataIntegrityViolationException.cla
ss)
        public ResponseEntity<ErrorResponse> handleSqlError(Exception e) {
            return ResponseEntity.status(HttpStatus.FORBIDDEN)
                    .body(new ErrorResponse("Database error", 40301));
        }


@ExceptionHandler(org.springframework.web.bind.MethodArgumentNotValidExceptio
n.class)
        public ResponseEntity<ErrorResponse> handleValidationError(Exception
e) {
            return ResponseEntity.status(HttpStatus.FORBIDDEN)
                    .body(new ErrorResponse("Validation error", 40301));
        }


    // 4. ВСЕ ОСТАЛЬНЫЕ ОШИБКИ -> 500
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAll(Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
```

```
                .body(new ErrorResponse(e.getMessage(), 50000));
    }
}
package org.example.newsapi.exception;

public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) {
        super(message);
    }
}
package org.example.newsapi.mapper;

import org.example.newsapi.dto.request.CommentRequestTo;
import org.example.newsapi.dto.response.CommentResponseTo;
import org.example.newsapi.entity.Comment;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;

@Mapper(componentModel = "spring")
public interface CommentMapper {

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "news", ignore = true) // Заполняется в CommentService
    Comment toEntity(CommentRequestTo request);

    @Mapping(target = "newsId", source = "news.id")
    CommentResponseTo toDto(Comment comment);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "news", ignore = true)
    void updateEntityFromDto(CommentRequestTo request, @MappingTarget Comment
comment);
}
package org.example.newsapi.mapper;

import org.example.newsapi.dto.request.MarkerRequestTo;
import org.example.newsapi.dto.response.MarkerResponseTo;
import org.example.newsapi.entity.Marker;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;

@Mapper(componentModel = "spring")
public interface MarkerMapper {

    @Mapping(target = "id", ignore = true)
    Marker toEntity(MarkerRequestTo request);

    MarkerResponseTo toDto(Marker marker);

    @Mapping(target = "id", ignore = true)
    void updateEntityFromDto(MarkerRequestTo request, @MappingTarget Marker
marker);
}
package org.example.newsapi.mapper;
```

```
import org.example.newsapi.dto.request.NewsRequestTo;
import org.example.newsapi.dto.response.NewsResponseTo;
import org.example.newsapi.entity.Marker;
import org.example.newsapi.entity.News;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;

import java.util.Set;
import java.util.stream.Collectors;

@Mapper(componentModel = "spring", imports = {Collectors.class, Set.class,
Marker.class})
public interface NewsMapper {

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "user", ignore = true)
    @Mapping(target = "markers", ignore = true)
    @Mapping(target = "created", ignore = true)
    @Mapping(target = "modified", ignore = true)
    @Mapping(target = "comments", ignore = true)
    News toEntity(NewsRequestTo request);

    @Mapping(target = "marker", ignore = true) // Чтобы MapStruct не искал
это поле
    @Mapping(target = "userId", source = "user.id")
    @Mapping(target = "markerIds", expression = "java(news.getMarkers() !=
null ?
news.getMarkers().stream().map(Marker::getId).collect(Collectors.toSet()) :
new java.util.HashSet<>())")
    NewsResponseTo toDto(News news);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "user", ignore = true)
    @Mapping(target = "markers", ignore = true)
    @Mapping(target = "created", ignore = true)
    @Mapping(target = "modified", ignore = true)
    @Mapping(target = "comments", ignore = true)
    void updateEntityFromDto(NewsRequestTo request, @MappingTarget News
news);
}
package org.example.newsapi.mapper;

import org.example.newsapi.dto.request.UserRequestTo;
import org.example.newsapi.dto.response.UserResponseTo;
import org.example.newsapi.entity.User;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;

@Mapper(componentModel = "spring")
public interface UserMapper {

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "news", ignore = true) // Игнорируем список новостей
при создании
    User toEntity(UserRequestTo request);
```

```
    UserResponseTo toDto(User user);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "news", ignore = true)
    void updateEntityFromDto(UserRequestTo request, @MappingTarget User
user);
}
package org.example.newsapi.repository;

import org.example.newsapi.entity.Comment;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface CommentRepository extends JpaRepository<Comment, Long> {
}
package org.example.newsapi.repository;

import org.example.newsapi.entity.Marker;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface MarkerRepository extends JpaRepository<Marker, Long> {
    boolean existsByName(String name);
    Optional<Marker> findByName(String name); // добавить этот метод
}
package org.example.newsapi.repository;

import org.example.newsapi.entity.News;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;
import org.springframework.stereotype.Repository;

@Repository
public interface NewsRepository extends JpaRepository<News, Long>,
JpaSpecificationExecutor<News> {
    boolean existsByTitle(String title);
}
package org.example.newsapi.repository;

import org.example.newsapi.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    // JpaRepository уже содержит методы findAll, findById, save, deleteById
    // Дополнительные методы можно объявлять здесь (например, findByLogin),
если понадобятся
    boolean existsByLogin(String login);
}
package org.example.newsapi.service;
```

```java
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.CommentRequestTo;
import org.example.newsapi.dto.response.CommentResponseTo;
import org.example.newsapi.entity.Comment;
import org.example.newsapi.entity.News;
import org.example.newsapi.exception.NotFoundException;
import org.example.newsapi.mapper.CommentMapper;
import org.example.newsapi.repository.CommentRepository;
import org.example.newsapi.repository.NewsRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class CommentService {

    private final CommentRepository commentRepository;
    private final NewsRepository newsRepository;
    private final CommentMapper commentMapper;

    @Transactional
    public CommentResponseTo create(CommentRequestTo request) {
        // Находим новость, к которой пишется комментарий
        News news = newsRepository.findById(request.getNewsId())
                .orElseThrow(() -> new NotFoundException("News not found with
id: " + request.getNewsId()));

        Comment comment = commentMapper.toEntity(request);
        comment.setNews(news);

        return commentMapper.toDto(commentRepository.save(comment));
    }

    public Page<CommentResponseTo> findAll(Pageable pageable) {
        return commentRepository.findAll(pageable)
                .map(commentMapper::toDto);
    }

    public CommentResponseTo findById(Long id) {
        return commentRepository.findById(id)
                .map(commentMapper::toDto)
                .orElseThrow(() -> new NotFoundException("Comment not found
with id: " + id));
    }

    @Transactional
    public CommentResponseTo update(Long id, CommentRequestTo request) {
        Comment comment = commentRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("Comment not found
with id: " + id));

        commentMapper.updateEntityFromDto(request, comment);

        // Если меняется привязка к новости (редкий кейс, но возможный)
```

```
        if (request.getNewsId() != null &&
!request.getNewsId().equals(comment.getNews().getId())) {
            News news = newsRepository.findById(request.getNewsId())
                    .orElseThrow(() -> new NotFoundException("News not
found"));
            comment.setNews(news);
        }

        return commentMapper.toDto(commentRepository.save(comment));
    }

    @Transactional
    public void delete(Long id) {
        if (!commentRepository.existsById(id)) {
            throw new NotFoundException("Comment not found with id: " + id);
        }
        commentRepository.deleteById(id);
    }
}
package org.example.newsapi.service;

import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.MarkerRequestTo;
import org.example.newsapi.dto.response.MarkerResponseTo;
import org.example.newsapi.entity.Marker;
import org.example.newsapi.exception.AlreadyExistsException;
import org.example.newsapi.exception.NotFoundException;
import org.example.newsapi.mapper.MarkerMapper;
import org.example.newsapi.repository.MarkerRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class MarkerService {

    private final MarkerRepository markerRepository;
    private final MarkerMapper markerMapper;

    @Transactional
    public MarkerResponseTo create(MarkerRequestTo request) {
        System.out.println(">>> ATTEMPTING TO CREATE MARKER WITH NAME: " +
request.getName());

        if (markerRepository.existsByName(request.getName())) {
            System.out.println(">>> MARKER ALREADY EXISTS: " +
request.getName());
            throw new AlreadyExistsException("Marker already exists");
        }

        Marker marker = markerMapper.toEntity(request);
        Marker saved = markerRepository.save(marker);

        System.out.println(">>> SUCCESSFULLY SAVED MARKER: " +
```

```
saved.getName() + " WITH ID: " + saved.getId());

        return markerMapper.toDto(saved);
    }

    public Page<MarkerResponseTo> findAll(Pageable pageable) {
        return markerRepository.findAll(pageable)
                .map(markerMapper::toDto);
    }

    public MarkerResponseTo findById(Long id) {
        return markerRepository.findById(id)
                .map(markerMapper::toDto)
                .orElseThrow(() -> new NotFoundException("Marker not found
with id: " + id));
    }

    @Transactional
    public MarkerResponseTo update(Long id, MarkerRequestTo request) {
        Marker marker = markerRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("Marker not found
with id: " + id));

        markerMapper.updateEntityFromDto(request, marker);
        return markerMapper.toDto(markerRepository.save(marker));
    }

    @Transactional
    public void delete(Long id) {
        if (!markerRepository.existsById(id)) {
            throw new NotFoundException("Marker not found with id: " + id);
        }
        markerRepository.deleteById(id);
    }
}
package org.example.newsapi.service;

import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.NewsRequestTo;
import org.example.newsapi.dto.response.NewsResponseTo;
import org.example.newsapi.entity.Marker;
import org.example.newsapi.entity.News;
import org.example.newsapi.entity.User;
import org.example.newsapi.exception.AlreadyExistsException;
import org.example.newsapi.exception.NotFoundException;
import org.example.newsapi.mapper.NewsMapper;
import org.example.newsapi.repository.MarkerRepository;
import org.example.newsapi.repository.NewsRepository;
import org.example.newsapi.repository.UserRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.List;
```

```java
import java.util.Set;

@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class NewsService {

    private final NewsRepository newsRepository;
    private final UserRepository userRepository;
    private final MarkerRepository markerRepository;
    private final NewsMapper newsMapper;

    @Transactional
    public NewsResponseTo create(NewsRequestTo request) {
        // 1. Проверяем пользователя
        if (request.getUserId() == null ||
!userRepository.existsById(request.getUserId())) {
            throw new NotFoundException("User not found");
        }

        // 2. Проверяем уникальность заголовка
        if (newsRepository.existsByTitle(request.getTitle())) {
            throw new AlreadyExistsException("News title already exists");
        }

        // 3. Создаём новость
        User user = userRepository.getReferenceById(request.getUserId());
        News news = newsMapper.toEntity(request);
        news.setUser(user);
        news.setCreated(LocalDateTime.now());
        news.setModified(LocalDateTime.now());

        // 4. Обрабатываем маркеры по именам
        if (request.getMarkerNames() != null &&
!request.getMarkerNames().isEmpty()) {
            Set<Marker> markers = new HashSet<>();
            for (String name : request.getMarkerNames()) {
                Marker marker = markerRepository.findByName(name)
                        .orElseGet(() -> {
                            // Создаём новый маркер, если не найден
                            Marker newMarker =
Marker.builder().name(name).build();
                            return markerRepository.save(newMarker);
                        });
                markers.add(marker);
            }
            news.setMarkers(markers);
        }

        News saved = newsRepository.saveAndFlush(news);
        return newsMapper.toDto(saved);
    }


    public Page<NewsResponseTo> findAll(Pageable pageable) {
        return newsRepository.findAll(pageable).map(newsMapper::toDto);
    }
```

```
    public NewsResponseTo findById(Long id) {
        return newsRepository.findById(id)
                .map(newsMapper::toDto)
                .orElseThrow(() -> new NotFoundException("News not found"));
    }

    @Transactional
    public NewsResponseTo update(Long id, NewsRequestTo request) {
        News news = newsRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("News not found"));

        // Проверяем, что пользователь существует
        if (!userRepository.existsById(request.getUserId())) {
            throw new NotFoundException("User not found");
        }

        // Обновляем поля новости (кроме маркеров)
        newsMapper.updateEntityFromDto(request, news);
        news.setUser(userRepository.getReferenceById(request.getUserId()));
        news.setModified(LocalDateTime.now());

        // Обрабатываем маркеры по именам
        if (request.getMarkerNames() != null) {
            Set<Marker> markers = new HashSet<>();
            for (String name : request.getMarkerNames()) {
                Marker marker = markerRepository.findByName(name)
                        .orElseGet(() -> {
                            // Создаём новый маркер, если не найден
                            Marker newMarker =
Marker.builder().name(name).build();
                            return markerRepository.save(newMarker);
                        });
                markers.add(marker);
            }
            news.setMarkers(markers);
        } else {
            // Если список имён не передан, можно оставить маркеры без
изменений
            // или очистить связь — зависит от требований
            // news.setMarkers(new HashSet<>());
        }

        return newsMapper.toDto(newsRepository.save(news));
    }
    @Transactional
    public void delete(Long id) {
        if (!newsRepository.existsById(id)) {
            throw new NotFoundException("News not found");
        }
        newsRepository.deleteById(id);
    }
}
package org.example.newsapi.service;

import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.UserRequestTo;
```

```java
import org.example.newsapi.dto.response.UserResponseTo;
import org.example.newsapi.entity.User;
import org.example.newsapi.exception.AlreadyExistsException;
import org.example.newsapi.exception.NotFoundException;
import org.example.newsapi.mapper.UserMapper;
import org.example.newsapi.repository.UserRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
@Transactional(readOnly = true) // По умолчанию транзакции только на чтение
public class UserService {

    private final UserRepository userRepository;
    private final UserMapper userMapper;

    @Transactional
    public UserResponseTo create(UserRequestTo request) {
        if (userRepository.existsByLogin(request.getLogin())) {
            throw new AlreadyExistsException("Login already exists"); //
Теперь это даст 403
        }
        User user = userMapper.toEntity(request);
        return userMapper.toDto(userRepository.save(user));
    }

    public Page<UserResponseTo> findAll(Pageable pageable) {
        return userRepository.findAll(pageable)
                .map(userMapper::toDto);
    }

    public UserResponseTo findById(Long id) {
        return userRepository.findById(id)
                .map(userMapper::toDto)
                .orElseThrow(() -> new NotFoundException("User not found with
id: " + id));
    }

    @Transactional
    public UserResponseTo update(Long id, UserRequestTo request) {
        User user = userRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("User not found with
id: " + id));

        userMapper.updateEntityFromDto(request, user);
        return userMapper.toDto(userRepository.save(user));
    }

    @Transactional
    public void delete(Long id) {
        if (!userRepository.existsById(id)) {
            throw new NotFoundException("User not found with id: " + id);
        }
        userRepository.deleteById(id);
```

```
    }
}
package org.example.newsapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class NewsApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(NewsApiApplication.class, args);
    }
}
server.port=24110
spring.datasource.url=jdbc:postgresql://localhost:5432/distcomp
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.liquibase.change-log=classpath:db/changelog/db.changelog-master.xml
spring.liquibase.enabled=true
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
        xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
        http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.24.xsd">

    <include file="changeset/v1.0.0-create-tables.xml"
relativeToChangelogFile="true"/>

</databaseChangeLog>
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
        xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
        http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.24.xsd">

    <changeSet id="1-create-sequences" author="brazhalovich">
        <createSequence sequenceName="user_seq" incrementBy="1"
startValue="1"/>
        <createSequence sequenceName="news_seq" incrementBy="1"
startValue="1"/>
        <createSequence sequenceName="marker_seq" incrementBy="1"
startValue="1"/>
        <createSequence sequenceName="comment_seq" incrementBy="1"
startValue="1"/>
    </changeSet>

    <changeSet id="2-create-tbl-user" author="brazhalovich">
        <createTable tableName="tbl_user">
            <column name="id" type="BIGINT"
defaultValueComputed="nextval('user_seq')"><constraints primaryKey="true"
```

```
nullable="false"/></column>
            <column name="login" type="VARCHAR(64)"><constraints
nullable="false" unique="true"/></column>
            <column name="password" type="VARCHAR(128)"><constraints
nullable="false"/></column>
            <column name="firstname" type="VARCHAR(64)"/><column
name="lastname" type="VARCHAR(64)"/>
        </createTable>
    </changeSet>

    <changeSet id="3-create-tbl-news" author="brazhalovich">
        <createTable tableName="tbl_news">
            <column name="id" type="BIGINT"
defaultValueComputed="nextval('news_seq')"><constraints primaryKey="true"
nullable="false"/></column>
            <column name="user_id" type="BIGINT"><constraints
nullable="false"/></column>
            <column name="title" type="VARCHAR(64)"><constraints
nullable="false" unique="true"/></column>
            <column name="content" type="VARCHAR(2048)"><constraints
nullable="false"/></column>
            <column name="created" type="TIMESTAMP"/><column name="modified"
type="TIMESTAMP"/>
        </createTable>
        <addForeignKeyConstraint baseTableName="tbl_news"
baseColumnNames="user_id" constraintName="fk_news_user"
referencedTableName="tbl_user" referencedColumnNames="id"
onDelete="CASCADE"/>
    </changeSet>

    <changeSet id="4-create-tbl-marker" author="brazhalovich">
        <createTable tableName="tbl_marker">
            <column name="id" type="BIGINT"
defaultValueComputed="nextval('marker_seq')"><constraints primaryKey="true"
nullable="false"/></column>
            <column name="name" type="VARCHAR(32)"><constraints
nullable="false" unique="true"/></column>
        </createTable>
    </changeSet>

    <changeSet id="5-create-tbl-comment" author="brazhalovich">
        <createTable tableName="tbl_comment">
            <column name="id" type="BIGINT"
defaultValueComputed="nextval('comment_seq')"><constraints primaryKey="true"
nullable="false"/></column>
            <column name="news_id" type="BIGINT"><constraints
nullable="false"/></column>
            <column name="content" type="VARCHAR(2048)"><constraints
nullable="false"/></column>
        </createTable>
        <addForeignKeyConstraint baseTableName="tbl_comment"
baseColumnNames="news_id" constraintName="fk_comment_news"
referencedTableName="tbl_news" referencedColumnNames="id"
onDelete="CASCADE"/>
    </changeSet>

    <changeSet id="6-create-tbl-news-marker" author="brazhalovich">
```

```
        <createTable tableName="tbl_news_marker">
            <column name="news_id" type="BIGINT"><constraints
nullable="false"/></column>
            <column name="marker_id" type="BIGINT"><constraints
nullable="false"/></column>
        </createTable>
        <addPrimaryKey tableName="tbl_news_marker" columnNames="news_id,
marker_id"/>
        <addForeignKeyConstraint baseTableName="tbl_news_marker"
baseColumnNames="news_id" constraintName="fk_nm_news"
referencedTableName="tbl_news" referencedColumnNames="id"
onDelete="CASCADE"/>
        <addForeignKeyConstraint baseTableName="tbl_news_marker"
baseColumnNames="marker_id" constraintName="fk_nm_marker"
referencedTableName="tbl_marker" referencedColumnNames="id"
onDelete="CASCADE"/>
    </changeSet>

    <changeSet id="7-insert-initial-user" author="brazhalovich">
        <insert tableName="tbl_user">
            <column name="id" valueNumeric="1"/>
            <column name="login" value="sashabrazhalovich2005@gmail.com"/>
            <column name="password" value="temp_password"/>
            <column name="firstname" value="Александр"/>
            <column name="lastname" value="Бражалович"/>
        </insert>
        <sql>SELECT setval('user_seq', 1);</sql>
    </changeSet>

    <changeSet id="8-insert-initial-markers" author="brazhalovich">
        <insert tableName="tbl_marker">
            <column name="id" valueNumeric="1"/>
            <column name="name" value="red2"/>
        </insert>
        <insert tableName="tbl_marker">
            <column name="id" valueNumeric="2"/>
            <column name="name" value="green2"/>
        </insert>
        <insert tableName="tbl_marker">
            <column name="id" valueNumeric="3"/>
            <column name="name" value="blue2"/>
        </insert>
        <!-- Обновляем последовательность, чтобы новые маркеры создавались с
правильными ID -->
        <sql>SELECT setval('marker_seq', 3);</sql>
    </changeSet>
</databaseChangeLog>
package org.example.newsapi;

import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
```

```
@Testcontainers
public abstract class AbstractIntegrationTest {

    // Определяем контейнер PostgreSQL (версия 15-alpine)
    @Container
    static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:15-alpine")
            .withDatabaseName("distcomp")
            .withUsername("postgres")
            .withPassword("postgres");

    // Динамически подменяем настройки application.properties на настройки
контейнера
    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", () -> postgres.getJdbcUrl() +
"?currentSchema=distcomp");
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);

        // Указываем Hibernate и Liquibase использовать схему по умолчанию
public (так проще в тестах)
        // или ту, что мы создали. Если скрипт Liquibase требует схему
distcomp,
        // нам нужно убедиться, что она создана.
        // Но TestContainers создает пустую БД.
        // Hibernate валидирует схему.
        // Проще всего переопределить схему на public для тестов,
        // либо добавить инициализирующий скрипт для создания схемы.

        // В данном случае мы используем URL с параметром
currentSchema=distcomp.
        // Postgres в тестконтейнере может не иметь этой схемы.
        // Поэтому добавим команду на создание схемы при старте контейнера:
        postgres.withInitScript("db/test-init-schema.sql");
    }
}
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>newsapi-parent</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>pom</packaging>

    <modules>
        <module>publisher</module>
        <module>discussion</module>
    </modules>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
```

```
        <version>3.2.2</version>
        <relativePath/>
    </parent>

    <properties>
        <java.version>17</java.version>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencyManagement>
        <dependencies>
            <!-- Общие зависимости можно определить здесь -->
        </dependencies>
    </dependencyManagement>

    <build>
        <pluginManagement>
            <plugins>
                <plugin>
                    <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-maven-plugin</artifactId>
                </plugin>


            </plugins>
        </pluginManagement>
    </build>
</project>
```